Executive verdict

DeployGuard is approximately **68% complete as a working product** and **58% ready for a repeatable FYP demonstration**.

The repository contains substantial, compiling implementations for all 17 original FYP modules and most of the two-lane architecture. Two real applications are currently healthy on AWS. However, the Railway-like architecture is **not the default active product path**:

- The local backend, Redis, configured PostgreSQL and workers are stopped.
- All normal-v1 planning/execution/consumer gates are unset and therefore off.
- The normal release worker accepts only an exact `production_canary` configuration.
- First-release UI stops after preparation instead of driving infrastructure plan/apply/release.
- The v1 deletion handler remains a placeholder.
- Security scanning is disabled/bypassed; FinOps is mock/bypassed.
- The developer UI still exposes Terraform approval, recovery, storage and orchestration-era controls.
- Current local durable state could not be correlated with AWS because the configured database is offline.

The largest gap is therefore **repeatable integration**, not missing backend architecture: one authenticated Overview action should reliably traverse canonical planning, exact outbox publication, a supervised worker, durable phase events, canonical state and refreshed UI under a safe local fixture and then under an approved production configuration.

Database/schema hardening is **not blocking the FYP demo or normal-v1 deployment paths**. It remains production-closure debt. `synchronize` is correctly disabled.

Document baseline

Both authoritative PDFs were found and completely extracted.

1. [Deploy Guard Updated Document Final.pdf](/home/hassan-sajjad/Deploy-Guard-dev/Deploy Guard Updated Document Final.pdf)
    - Title: “Project Proposal & Scope Document — DeployGuard: A Reliable Repository Scanning and Deployment Orchestration System”
    - Version: no numbered version appears in the document; filename and contents identify it as the updated final proposal.
    - Contains modules 6.1–6.17.
2. [DeployGuard_Final_Railway_Like_Architecture_and_Recovery_Report.pdf](/home/hassan-sajjad/Deploy-Guard-dev/DeployGuard_Final_Railway_Like_Architecture_and_Recovery_Report.pdf)

    - Title: “DeployGuard Railway-Like Two-Lane PaaS Architecture — Complete deployment, failure containment, rollback, reconciliation, deletion, UI, and release-readiness specification”
    - Version: **3.0, July 2026**.

I recursively inventoried 67 documentation files. The most important current records are:

- [Phase 10 integration record](/home/hassan-sajjad/Deploy-Guard-dev/docs/architecture/two-lane-migration/PHASE_10_NORMAL_WORKFLOW_INTEGRATION.md)
- [TypeORM schema audit](/home/hassan-sajjad/Deploy-Guard-dev/docs/architecture/two-lane-migration/TYPEORM_SCHEMA_CONTRACT_AUDIT.md)
- [Current orchestration map](/home/hassan-sajjad/Deploy-Guard-dev/docs/architecture/two-lane-migration/CURRENT_ORCHESTRATION_MAP.md)
- [Setup guide](/home/hassan-sajjad/Deploy-Guard-dev/docs/SETUP_REQUIRED.md)

Documentation is not internally consistent:

- `CURRENT_ORCHESTRATION_MAP.md` predates the retirement of legacy mutation routes.
- `SETUP_REQUIRED.md` still describes older combined-pipeline behavior.
- Phase 10 contains strong historical local/live evidence, but several sections are chronological rather than authoritative summaries.
- The TypeORM audit still contains contradictory historical totals—61 versus 26 migration-only foreign keys—so it cannot support a closure claim.

The current workspace also has **476 modified or untracked paths**. This assessment describes the present workspace, not a clean reproducible commit.

Architecture reconciliation

The FYP functional outcomes remain authoritative for authentication, projects, detection, validation, security, cost governance, provisioning, state, storage, releases, monitoring, AI, notifications, deletion, export and billing.

Version 3.0 supersedes these implementation details:

- One combined Terraform-driven deployment pipeline → separate release, infrastructure and deletion lanes.
- Terraform on every push → ordinary release-only commits bypass Terraform.
- Legacy `ProjectPipelineRun` as deployment truth → deployment intents, manifests, outbox, journals and canonical state.
- DynamoDB-centric locking → PostgreSQL ownership/fencing plus remote-state/native Terraform locking.
- Latest attempted deployment as live state → separate latest candidate and verified stable release.
- Generic always-on workers → versioned, default-off, exact-scope consumers.
- Developer-controlled recovery stages → platform-owned reconciliation and bounded rollback.

- Old multi-step cleanup UI → intended one-click deletion state machine.
- Developer Terraform/recovery pages → intended simplified Overview and Pipeline.

Superseded requirements should not be counted as missing. The problem is that several legacy presentation and read paths remain visible while their write endpoints have been retired.

Original FYP module matrix

`Iso` means a current focused or deterministic test passed. `Local` means real local-service evidence; "historical" means Phase 10 reports it but this assessment could not reproduce it with the configured services stopped.

Module	Final status	Current implementation and legacy position	B/F	Runtime	Iso	Local	AWS
6.1 Authentication/audit	Implemented and locally verified	Email/password, GitHub OAuth, cookie sessions, RBAC and persisted sanitized audit events. Audit writes are best-effort rather than DB-immutable.	Y/Y	App off	Yes	Not current	N/A
6.2 Projects/workspaces	Implemented but not operationally proven	Project ownership, environment identity, repository configuration, admin/developer/readonly access and workspace summaries exist.	Y/Y	App off	Partial	Not current	Current authenticated mul user/project smok [Projects](/home/ ssan-sajjad/Deplo Guard-dev/backer /src/projects/ projects.service.ts
6.3 Stack detection	Implemented and locally verified	Deterministic Node/Python detection, monorepo selection, Dockerfile fallback and unsupported-stack blocking.	Y/Y	App off	Yes	Fixture only	N/A
6.4 Templates/preflight	Implemented and locally verified	Templates, deployment contracts, environment requirements and preflight snapshots are wired.	Y/Y	App off	Yes	Fixture only	Reproduce agains one real connecte repository. [Preflight](/home/ assan-sajjad/Depl y-Guard-dev/back nd/src/projects/ templates/ preflight.service.t
6.5 CI/CD and BullMQ	Implemented but disabled	Exact v1 outbox/BullMQ exists; legacy pipeline mutations return 410.	Y/Y	Gates/ workers off	Yes	Historical	Partial

Module	Final status	Current implementation and legacy position	B/F	Runtime	Iso	Local	AWS
		GitHub Actions is optional rather than the authoritative executor.					
6.6 Security	Partially implemented	Trivy adapters, findings, policy, approvals and UI exist, but normal-v1 release handlers do not invoke the security service. Current config has scanning off and bypass enabled.	Y/Y	Disabled/ bypass	Yes	No	No
6.7 FinOps	Partially implemented	Infracost, policies, cost records and UI exist. V1 infrastructure-plan review uses Infracost, but current config is mock/bypass and normal release does not use it.	Y/Y	Mock/ bypass	Partial	Historical	No
6.8 Provisioning/discovery	Implemented but disabled	Real Terraform plan/apply services, managed networking/database, ECR/ECS/ALB/Cloud Map outputs and exact workers exist.	Y/Y	Gates off	Yes	Historical	Yes
6.9 Distributed state	Implemented but disabled	S3 state/versioning, saved plans, state identity, native lock and PostgreSQL ownership/leases supersede DynamoDB design.	Y/Y	App/ workers off	Yes	Historical	Partial
6.10 Storage/backup	Implemented but not operationally proven	EFS, backup records, restore controls and UI exist; Smart Retail has a healthy persistent managed DB foundation.	Y/Y	App off	Partial	Historical	Partial
6.11 Orchestration/rollback	Implemented but disabled	Fenced release handler, task registration, service update, convergence,	Y/Y	Release worker off	Yes	Historical	Yes

Module	Final status	Current implementation and legacy position	B/F	Runtime	Iso	Local	AWS
		stable promotion and rollback logic exist. Legacy orchestration reads remain.					
6.12 Monitoring	Partially implemented	CloudWatch logs/metrics, SSE, health, pipeline metrics and UI exist. Prometheus is disabled and the live observability UI was not exercised.	Y/Y	App off; Prometheus off	Partial	No	ECS/ALB only
6.13 AI diagnosis	Implemented but not operationally proven	Gemini adapter, preprocessing, structured response validation, rate limits, conversation persistence and UI exist. It still depends on legacy `ProjectPipelineRun` evidence.	Y/Y	Provider configured; app off	Yes	Provider not called	N/A
6.14 Notifications	Implemented but disabled	SNS subscription/delivery/dedup/retry and UI settings exist. SNS topic is unconfigured and v1 lifecycle integration is incomplete.	Y/Y	Disabled	Yes	No	No
6.15 Deletion	Partially implemented	Older dual-confirmation destroy, inventory and cleanup system is extensive. V1 deletion envelope/ownership exists, but its handler is still a success placeholder.	Y/Y	App off	Logic only	No	No
6.16 Terraform export	Implemented and locally verified	Deterministic ZIP creation, authenticated API, entitlement/audit integration and UI download exist.	Y/Y	App off	Yes	Deterministic local	N/A
6.17 Billing	Implemented but disabled	Stripe checkout/webhooks/portal/invoices, plans, usage records and UI exist. Stripe is unconfigured and enforcement disabled.	Y/Y	Disabled	Yes	No	N/A

Strict unweighted count:

- Fully complete against the entire original specification: **0/17**
- Substantively implemented: **11/17**
- Partial or materially disconnected: **6/17**
- Pure documentation-only or wholly missing: **0/17**

Two-lane subsystem matrix

Subsystem	Status	Evidence and remaining boundary
Semantic classifier/planner	Implemented and locally verified	Immutable classification and transactional planners exist; current authenticated DB run was unavailable. [Planner](/home/hassan-sajjad/Deploy-Guard-dev/backend/src/orchestration-contracts/planner/transactional-deployment-planner.service.ts)
Infrastructure/release manifests	Implemented but not operationally proven	Revisioned immutable entities and validators exist; current durable records could not be inspected.
Exact outbox dispatch	Implemented and locally verified	`dispatchExact()` locks one row and produces a deterministic job.
Ownership/fencing/leases	Implemented and locally verified	Cross-lane ownership, operation leases, heartbeat, terminal outbox and side-effect journals are substantial and focused tests pass. [Ownership](/home/hassan-sajjad/Deploy-Guard-dev/backend/src/orchestration-contracts/worker-runtime/inactive-v1-pre-execution-ownership.service.ts)
Later release lane	Implemented but disabled	UI prepares and dispatches later releases; Project-Tst is presently healthy on task-definition revision 6. Worker gates are off.
Managed first release	Partially implemented	Backend preparation and supervised execution exist and Smart Retail is live, but Overview intentionally stops after first-release preparation.
Infrastructure plan	Implemented but disabled	Exact route, queue, consumer, saved plan and continuation exist. Not invoked by Overview and not Compose-packaged.
Infrastructure apply	Implemented but disabled	Exact route, consumer, saved-plan-only apply and reconciliation exist. Not invoked by Overview and not Compose-packaged.
Deletion lane	Documentation/scaffolding only	Envelope and ownership safety exist; the actual v1 handler is a placeholder.
Release reconciliation/rollback	Implemented but disabled	Durable journals, uncertain-outcome reconciliation and bounded convergence exist; current process is off.
Canonical current state	Implemented but not operationally proven	Overview/Dashboard use it, but several pages also make direct pipeline/orchestration/observability calls. [Current state](/home/hassan-sajjad/Deploy-Guard-dev/backend/src/projects/current-state/project-current-state.service.ts)
Stable vs latest release	Implemented but not operationally proven	V1 history/stable projections coexist with legacy history and are deduplicated; current DB could not be queried.
Supervised	Partially	Release worker has Compose health/restart handling. Infrastructure

Subsystem	Status	Evidence and remaining boundary
workers	implemented	workers are package commands only. All are default-off. [Compose](/home/hassan-sajjad/Deploy-Guard-dev/docker-compose.yml)
		Overview has v1 prepare/dispatch, but still exposes apply approval, retry/cancel, Terraform details and advanced infrastructure/recovery routes.
Railway-like developer UI	Partially implemented	[Overview](/home/hassan-sajjad/Deploy-Guard-dev/frontend/src/pages/ProjectDetails.jsx), [routes](/home/hassan-sajjad/Deploy-Guard-dev/frontend/src/routes/AppRoutes.jsx), [sidebar](/home/hassan-sajjad/Deploy-Guard-dev/frontend/src/components/layout/Sidebar.jsx)

Current runtime path

The intended later-release path is:

```
Overview Start
  → POST /api/projects/:id/release-lane/prepare
  → transactional planner
  → DeploymentIntent + candidate ReleaseManifest + pending outbox
  → POST /release-lane/dispatch
  → dispatchExact()
  → deployguard-release-v1 BullMQ job
  → independently supervised release consumer
  → ownership + operation lease + fenced handler
  → Docker/ECR/ECS journals and reconciliation
  → stable ReleaseManifest
  → ProjectCurrentStateService
  → refreshed Overview/Pipeline/Dashboard
```

The path in the current checkout stops earlier:

```
No backend / Redis / configured PostgreSQL / worker process
  → v1 gates unset
  → preparation returns disabled if the app is started as configured
  → no outbox publication
  → release consumer exits with NORMAL_V1_RELEASE_CONSUMER_DISABLED
```

For first release, even with planning enabled, Overview stops deliberately after preparation and says infrastructure execution is not enabled. The exact infrastructure plan/apply routes are not chained from the UI.

First broken boundary for each required flow

Flow	First unproven or broken boundary
1. Signup/login/audit	Real current HTTP → PostgreSQL session/audit flow; only focused contract verification ran.
2. GitHub/project creation	Current OAuth callback and database persistence; app/database are stopped.

Flow	First unproven or broken boundary
3. Detection/preflight	Authenticated repository checkout in the current runtime; deterministic fixtures pass.
4. Deploy command	Default-off planning gate.
5. Semantic classification	Current database transaction; pure/planner checks exist.
6. Release-only	Consumer activation and current PostgreSQL/Redis runtime.
7. First deployment	Overview does not dispatch infrastructure plan/apply.
8. Docker build and scan	V1 path does not invoke the Trivy gate; configured scan is disabled.
9. ECR push	Current worker activation; production client exists and AWS repositories exist.
10. Terraform plan/apply	UI handoff and independently supervised infrastructure workers.
11. ECS/ALB/stable promotion	Current durable-to-AWS reconciliation; AWS itself is healthy.
12. Canonical state to UI	Configured PostgreSQL is unavailable; some pages still query secondary APIs.
13. Redeploy without Terraform	Worker/gate activation. Code and Project-Tst history support the design.
14. Rollback	A current candidate failure/eligibility proof; only logic and historical evidence exist.
15. Monitoring/log streaming	Running backend plus CloudWatch/SSE request; Prometheus is off.
16. AI diagnosis	Canonical v1 failure evidence and a live Gemini call.
17. Notification	SNS topic/subscription configuration and canonical v1 event emission.
18. One-click deletion	Actual v1 deletion handler; current handler is a placeholder.
19. Terraform export	Current authenticated HTTP download; deterministic ZIP logic passes.
20. Billing	Stripe configuration and enforcement; both are off.

Legacy versus normal-v1

Runtime-active now

- No backend, Vite frontend, BullMQ worker or normal-v1 consumer is running.
- No Docker Compose service is active.
- Configured PostgreSQL expects port 5433; nothing listens there.
- Redis is not listening on 6379.
- Externally, the existing AWS application services remain active.

Default-off normal-v1 components

- Normal release planning and execution.
- Release consumer and production client.
- Infrastructure plan dispatch/consumer.
- Infrastructure apply dispatch/consumer.

- First-release live execution.
- Automatic convergence/reconciliation.
- Cross-lane ownership rollout/enforcement.
- Exact project/environment allowlists.

The release consumer additionally rejects fixture mode and requires `production_canary` plus one exact UUID and `dev` scope. See [runtime gate](/home/hassan-sajjad/Deploy-Guard-dev/backend/src/orchestration-contracts/worker-runtime/normal-v1-release-consumer-runtime.service.ts).

Frontend-reachable

- Normal first/later release preparation.
- Later-release exact dispatch.
- Canonical current state and release history.

Backend-only or unreachable from normal UI

- Infrastructure-plan exact dispatch.
- Infrastructure-apply exact dispatch.
- Infrastructure worker health.
- Apply-child approval lifecycle.
- V1 deletion execution.

Legacy code still used

- Historical `ProjectPipelineRun` reads and pipeline events.
- AI troubleshooting keyed to pipeline runs.
- Orchestration, logs, metrics, storage, state, rollback and security pages.
- Legacy stable-deployment compatibility projection.
- Cleanup/inventory/state recovery as admin/internal functions.

Legacy deployment mutation endpoints now correctly return `410 LEGACY_DEPLOYMENT_EXECUTION_RETIRED`.

Conflicting legacy UI

Overview and Pipeline still import or render:

- Terraform apply approval.
- Legacy retry/cancel.
- Technical Terraform/FinOps/state modes.
- Recovery-stage controls.
- Direct pipeline history.
- Advanced Infrastructure, State, Storage, Orchestration and Recovery routes.

These do not necessarily re-enable retired mutations, but they contradict the version-3 developer contract.

There is also compatibility debt in [NormalReleaseLaneExecutionService](/home/hassan-sajjad/Deploy-Guard-dev/backend/src/orchestration-contracts/release-lane/normal-release-lane-execution.service.ts): several blocked results still set `fallbackToLegacy: true`, although the frontend ignores that field and actual legacy mutation routes are retired.

Verification performed

Current non-mutating checks passed:

- Backend `npx tsc --noEmit`
- Backend Nest build
- Frontend Vite build
- Authentication
- Stack detection across 14 framework fixtures
- Security policy
- Notifications
- Billing/entitlements
- Terraform export
- Normal release planning
- Normal current-state projection
- Normal consumer lifecycle
- Normal UI handoff/current-state UI guards
- AI troubleshooting
- Rollback shadow logic
- `git diff --check`

These are focused contract/fixture checks. They are not a current PostgreSQL/Redis supervised integration run.

Read-only AWS checks proved:

- Authenticated identity in the expected account and `us-east-1`.
- Project-Tst service: ACTIVE, `1/1/0`, PRIMARY rollout completed, task definition revision 6.
- Smart Retail application: ACTIVE, `1/1/0`, PRIMARY rollout completed, task definition revision 3.
- Smart Retail managed PostgreSQL service: ACTIVE, `1/1/0`.
- Both ALB target groups report healthy targets.
- Project-Tst root returns HTTP 200 with a Streamlit page.
- Smart Retail root returns HTTP 200 with the expected application title.

No AWS mutation occurred. Current PostgreSQL release identities, provenance and ownership cleanup could not be reverified because the configured database is offline.

Completion calculations

Scores use equivalent completion points: 1.0 for implemented and operationally proven, 0.8 for implemented with local verification, 0.6 for implemented but disabled/disconnected, 0.3 for partial/scaffolding and 0 for missing. Documentation alone earns zero.

Measurement	Result	Numerator/denominator and method	Confidence
Original FYP scope	**72%**	72.2/100. Criticality weights: auth 6, projects 5, detection 5, preflight 5, CI 9, security 7, FinOps 6, infrastructure 10, state 7, storage 5, orchestration 10, observability 6, AI 4, notifications 3, deletion 6, export 3, billing 3.	High
Two-lane implementation	**75%**	10.5 equivalent subsystems / 14, equally weighted.	High
Backend implementation	**82%**	24.5/30 equal capability checks: 17 FYP backends plus 13 core two-lane contracts.	High
Frontend integration	**66%**	10.5/16 user-visible capability checks. Advanced pages exist, but canonical v1 integration and simplification are incomplete.	High
Local operational verification	**43%**	8.5/20 runtime flows: 4 current full focused proofs, 9 partial/historical fixture proofs, 7 currently unproven.	Medium-low
Live AWS verification	**56%**	5/9 cloud checkpoints: identity, foundation presence, ECS rollout, ALB health and HTTP verified; current durable mapping, exact state/provenance, DB persistence and rollback/deletion remain unverified.	Medium
FYP demonstration readiness	**58%**	7 equivalent /12 demo checkpoints. Core pages and modules compile, but the canonical run, security/cost gates, deletion and provider-backed modules are not ready as one script.	Medium
Production-readiness evidence	**43%**	6/14 operational controls. Fencing/default-off safety is strong; multi-project supervision, mandatory gates, audit guarantees, deletion and continuous reconciliation are not.	Medium
Overall working product	**68%**	25% FYP + 25% two-lane + 15% backend + 10% frontend + 10% local operations + 10% AWS + 5% production readiness. Numerator 67.6/100.	Medium-high

Deduplicated remaining backlog

FYP/demo critical

Task and outcome	Affected subsystems	Why/ prerequisite	Scope	Size/ model	Prompts	Acceptance evidence
Safe local canonical operation	Release runtime, fixture adapters, Compose/scripts, PG/Redis	Existing worker rejects fixture mode; prerequisite for repeatable demonstration.	Local	Medium, **Sol High**	2	Ready worker, exact job, one handler call, durable events, canonical API, clean shutdown.
Canonical phases and simplified Overview/Pipeline	Current-state types/service, Overview, Pipeline, routes/sidebar	Current UI mixes legacy stages and controls. Depends on canonical operation.	Local	Large, **Terra High**	2	Analyze/Build/Prepare/ Deploy/Verify/Live or Failed survives refresh; legacy controls absent.
Integrate Trivy and cost governance into v1	Security, release handler, infrastructure plan review, current state	Current production defaults bypass both FYP gates.	Local then AWS read-only	Large, **Sol High**	2	Blocking and passing fixtures; no promotion before approved policy.
Complete first-release UI handoff	Overview, plan/apply dispatch, approval state, three consumers	UI currently stops at prepared. Depends on canonical phases and guarded providers.	Local	Large, **Sol High**	2–3	One UI action reaches apply-awaiting-approval, approved apply, then release without duplicates.
Implement v1 one-click deletion	Deletion planner/outbox/consumer/current state/UI	Version-3 deletion is still a placeholder.	Local, later AWS	Large, **Sol High**	3	One exact deletion intent, protected data policy, reconciliation and live disposable teardown.

Before the next repeatable first AWS deployment

A successful first AWS deployment already exists for Smart Retail. To make it repeatable for another project:

Task	Scope	Size/model	Prompts	Required evidence
Package/supervise	Local →	Medium,	2	Health, restart, exact scope, natural

Task	Scope	Size/model	Prompts	Required evidence
infrastructure plan and apply workers under the repository convention	AWS	**Terra High**		stop and no duplicate plan/apply.
Run one new exact-scope first release through the UI	AWS	Large, **Sol High**	1–2	Browser network evidence, one plan/apply/image/service, state/provenance match, ALB/HTTP and cleanup.

Before final submission

Task	Scope	Size/model	Prompts	Required evidence
Current real-service FYP regression pack	Local PG/Redis/browser	Medium, **Terra High**	2	Auth, project, detect, preflight, release, state, refresh, export, AI/notification disabled truth.
Observability/provider demonstrations	Local/AWS read-only plus configured sandbox providers	Medium, **Terra High**	2	CloudWatch logs/metrics, one AI result, one notification or explicit unavailable state.
Consolidate setup/evidence and produce a clean reproducible commit	Docs/config	Small, **Luna Medium**	1	Stale orchestration/setup claims removed; exact startup/demo commands; clean diff ownership.

Production hardening

Task	Scope	Size/ model	Prompts	Evidence
Replace one-project canary scoping with controlled multi-project supervision	Production architecture	Large, **Sol High**	3	Multi-scope isolation, capacity/restart testing and no cross-project claims.
Harden audit/session/request security	Backend/ security	Large, **Sol High**	2	Durable audit guarantees, CSRF/session tests, secret and denial-path evidence.
Finish schema-contract classification	Disposable PostgreSQL only	Large, **Sol High**	2	Generated reconciliation, no unexplained ambiguity, clone integrity and replay. This does not block the FYP demo.
Continuous reconciliation and operational alerting	Workers/ observability	Large, **Sol High**	2–3	Crash/restart/uncertain-side-effect soak with alerts and no duplicate mutations.

Optional or reduced by version 3

- GitHub Actions as the authoritative deployment executor: superseded; retain only as optional validation.
- DynamoDB Terraform locking: superseded by the v3 PostgreSQL/S3-native model.
- Developer-controlled Terraform stage retry: superseded and should not return.

- Multi-cloud support: outside the original declared scope.
- Fully live Stripe billing for the FYP: useful, but a truthful “provider not configured” state can be acceptable if scope is explicitly reduced.

Remaining critical path

To reach one successful local canonical normal-v1 operation:

1. Add the explicit production-safe local executor boundary.
2. Start disposable PostgreSQL/Redis and one supervised worker.
3. Dispatch one authenticated exact outbox.
4. Persist canonical phase events.
5. render those phases on Overview and verify refresh.

To reach a repeatable new first AWS deployment:

1. Complete the first-release UI plan/apply/release handoff.
2. Package or supervise all three exact consumers consistently.
3. Integrate mandatory security and cost decisions.
4. Run one new exact-scope deployment and reconcile before retrying.

An ordinary Terraform-free redeploy already exists in code and has historical Project-Tst live evidence. Repeating it requires only exact gate/worker activation after the local proof.

A verified stable live URL already exists for Smart Retail and Project-Tst in AWS. What is missing is current durable-database correlation and a repeatable product-level execution.

Prompt estimate

- **Best case: 10 bounded prompts**
  Assumes existing fixtures need little repair, AWS identities remain healthy, provider-backed modules may be demonstrated as explicitly disabled, and v1 deletion is deferred from the demo.
- **Realistic FYP-ready case: 16 bounded prompts**
  Includes local canonical operation, phase/UI integration, security/FinOps, first-release handoff, deletion, real-service regression and documentation consolidation.
- **Worst case: 24 bounded prompts**
  Assumes AWS exposes task, state, secret, health or rollback defects and one or more supervised restart/reconciliation packages are needed.
- **Production hardening beyond FYP readiness: another 7–10 prompts**
  Mainly multi-project supervision, audit/security guarantees, schema closure and continuous operational reconciliation.

Final truth statement

Mostly implemented but not integrated.

The judgment is supported by concrete evidence:

- Backend and frontend compile.
- Focused implementation checks pass.
- Real AWS services and URLs are currently healthy.
- Core two-lane planning, dispatch, ownership, infrastructure and release services are substantial.
- Legacy mutation endpoints are retired.

But the default checkout runs no application or worker, every normal-v1 activation gate is off, first release stops in the UI, security/cost enforcement is bypassed, v1 deletion is a placeholder, and the developer experience still exposes superseded controls. The Railway-like architecture is therefore real and previously exercised, but it is not yet the ordinary, repeatable, end-to-end product path.